Homework 2: Training Multilayer Perceptrons

Qinglang Wang (Andrew ID: qinglang)

February 8, 2026

Deliverable 2: Linear Separable

Configs and Training Logs

```
Start training with configs:
  model_config:
    layer_nodes: [2, 1, 1]
    optimizer: SGD(lr=0.2)
    hidden_act: tanh
    output_act: sigmoid
  training_config:
    epochs: 1000
    criterion: <class 'loss.MSELoss'>
  trainset_config:
    data: <ndarray shape=(200, 2)>
    labels: <ndarray shape=(200, 1)>
    batch_size: 100
    shuffle: True
  validation_config:
    count_correct: <function count_binary_correct at 0x...>
    mute: True
  valset_config:
    data: <ndarray shape=(200, 2)>
    labels: <ndarray shape=(200, 1)>
    batch_size: 100
    shuffle: False
MLP(
  (0): Linear(in_features=2, out_features=1)
  (1): Tanh()
  (2): Linear(in_features=1, out_features=1)
  (3): Sigmoid()
)
Training done, with highest validation accuracy of 0.98.
```

The best validation accuracy achieved is **98%**.

Visualizations

Figure 1 shows the loss curve and decision boundary.

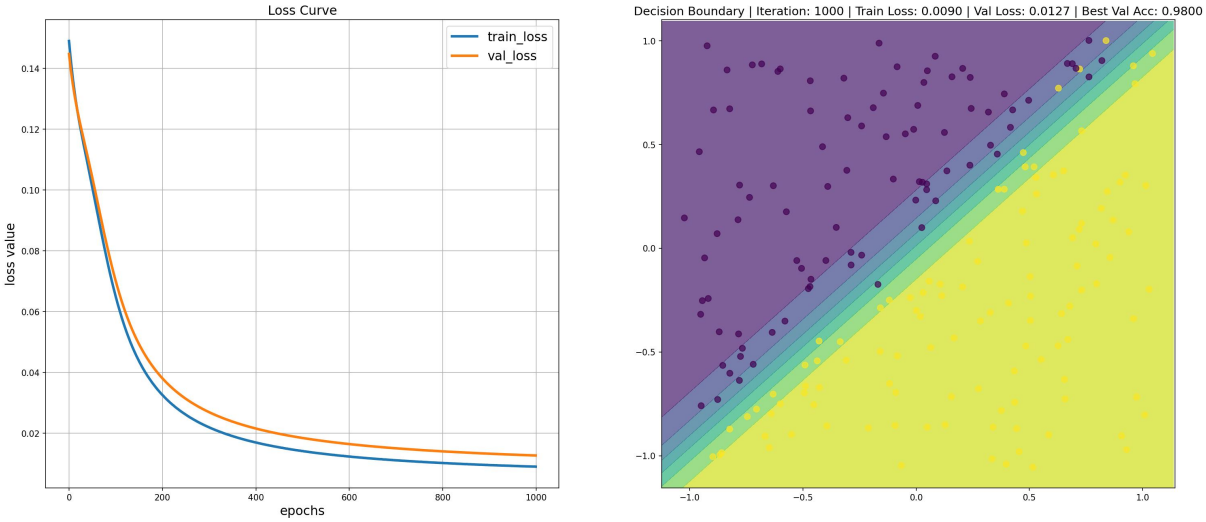

Figure 1: Training results for Linear Separable dataset.

Deliverable 3: XOR Problem

Configs and Training Logs

```
Start training with configs:
  model_config:
    layer_nodes: [2, 4, 1]
    optimizer: Adam(lr=0.02, betas=(0.9, 0.999), eps=1e-09)
    hidden_act: tanh
    output_act: sigmoid
  training_config:
    epochs: 500
    criterion: <class 'loss.BCELoss'>
  trainset_config:
    data: <ndarray shape=(200, 2)>
    labels: <ndarray shape=(200, 1)>
    batch_size: 100
    shuffle: True
  validation_config:
    count_correct: <function count_binary_correct at 0x000001AE3228A5E0>
    mute: True
  valset_config:
    data: <ndarray shape=(200, 2)>
    labels: <ndarray shape=(200, 1)>
    batch_size: 200
    shuffle: False
MLP(
  (0): Linear(in_features=2, out_features=4)
  (1): Tanh()
  (2): Linear(in_features=4, out_features=1)
  (3): Sigmoid()
)
Training done, with highest validation accuracy of 0.98.
```

The best validation accuracy achieved is **98%**.

Visualizations

Figure 2 shows the loss curve and decision boundary.

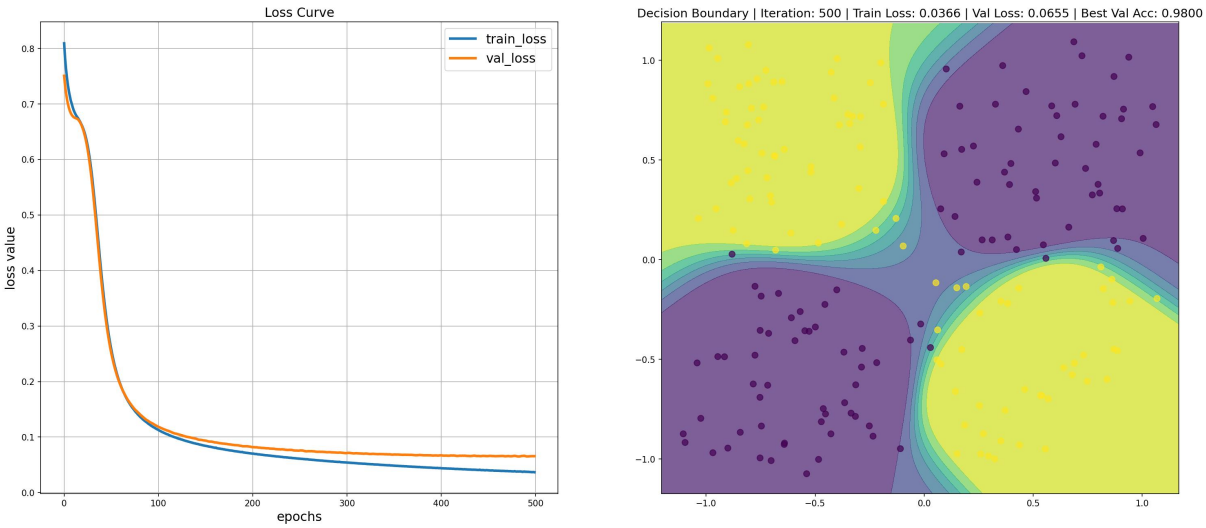

Figure 2: Training results for XOR dataset.

Deliverable 4: Differences in Cost Function

L2 Loss

Configs and Training Logs

```
Start training with configs:
  model_config:
    layer_nodes: [2, 10, 1]
    optimizer: Adam(lr=0.01, betas=(0.9, 0.999), eps=1e-09)
    hidden_act: relu
  training_config:
    epochs: 300
    criterion: <class 'loss.MSELoss'>
  trainset_config:
    data: <ndarray shape=(200, 2)>
    labels: <ndarray shape=(200, 1)>
    batch_size: 100
    shuffle: True
  validation_config:
    count_correct: <function count_binary_correct at 0x00000181FE43AD30>
    mute: True
  valset_config:
    data: <ndarray shape=(200, 2)>
    labels: <ndarray shape=(200, 1)>
    batch_size: 200
    shuffle: False
MLP(
  (0): Linear(in_features=2, out_features=10)
  (1): ReLU()
  (2): Linear(in_features=10, out_features=1)
)
```

```
Training done, with highest validation accuracy of 0.975.
```

The best validation accuracy achieved is **97.5%**.

Visualizations

Figure 3 shows the loss curve and the decision boundary.

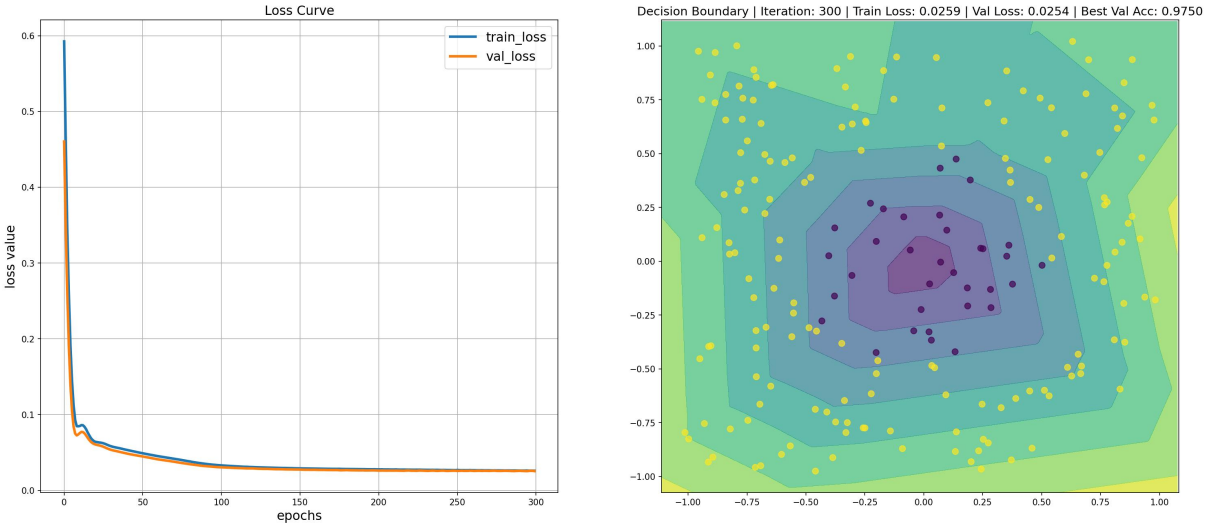

Figure 3: Training results for L2 Loss function on Circle dataset.

CE Loss

Configs and Training Logs

```
Start training with configs:
  model_config:
    layer_nodes: [2, 10, 1]
    optimizer: Adam(lr=0.01, betas=(0.9, 0.999), eps=1e-09)
    hidden_act: relu
  training_config:
    epochs: 300
    criterion: <class 'loss.BCEFromLogitsLoss'>
  trainset_config:
    data: <ndarray shape=(200, 2)>
    labels: <ndarray shape=(200, 1)>
    batch_size: 100
    shuffle: True
  validation_config:
    count_correct: <function count_binary_correct_from_logits at 0x00000181FFED7940>
    mute: True
  valset_config:
    data: <ndarray shape=(200, 2)>
    labels: <ndarray shape=(200, 1)>
    batch_size: 200
    shuffle: False
MLP(
  (0): Linear(in_features=2, out_features=10)
  (1): ReLU()
  (2): Linear(in_features=10, out_features=1)
```

```
)
Training done, with highest validation accuracy of 0.975.
```

The best validation accuracy achieved is **97.5%**.

Visualizations

Figure 4 shows the loss curve and the decision boundary.

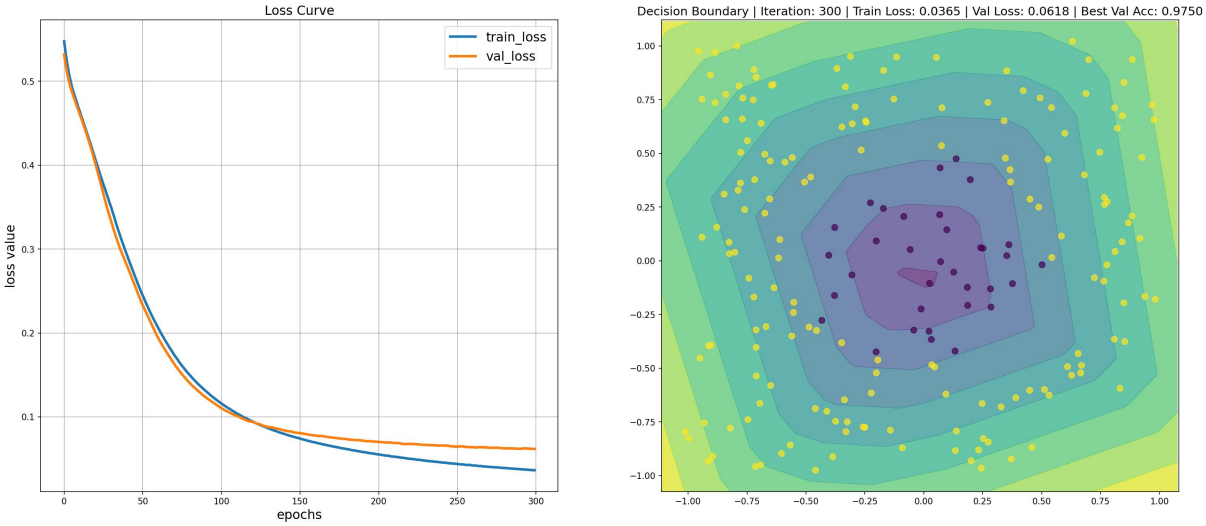

Figure 4: Training results for CE Loss function on Circle dataset.

Deliverable 5: Differences in Optimizers

Vanilla SGD

Configs and Training Logs

```
Start training with configs:
  model_config:
    layer_nodes: [2, 32, 16, 16, 1]
    optimizer: SGD(lr=0.02)
    hidden_act: relu
  training_config:
    epochs: 9000
    criterion: <class 'loss.BCEFromLogitsLoss'>
  trainset_config:
    data: <ndarray shape=(200, 2)>
    labels: <ndarray shape=(200, 1)>
    batch_size: 64
    shuffle: True
  validation_config:
    count_correct: <function count_binary_correct_from_logits at 0x00000229775789D0>
    mute: True
  valset_config:
    data: <ndarray shape=(200, 2)>
    labels: <ndarray shape=(200, 1)>
    batch_size: 200
    shuffle: False
```

```
MLP(
  (0): Linear(in_features=2, out_features=32)
  (1): ReLU()
  (2): Linear(in_features=32, out_features=16)
  (3): ReLU()
  (4): Linear(in_features=16, out_features=16)
  (5): ReLU()
  (6): Linear(in_features=16, out_features=1)
)
Training done, with highest validation accuracy of 0.93.
```

The best validation accuracy achieved is **93%**.

Visualizations

Figure 5 shows the loss curve and the decision boundary.

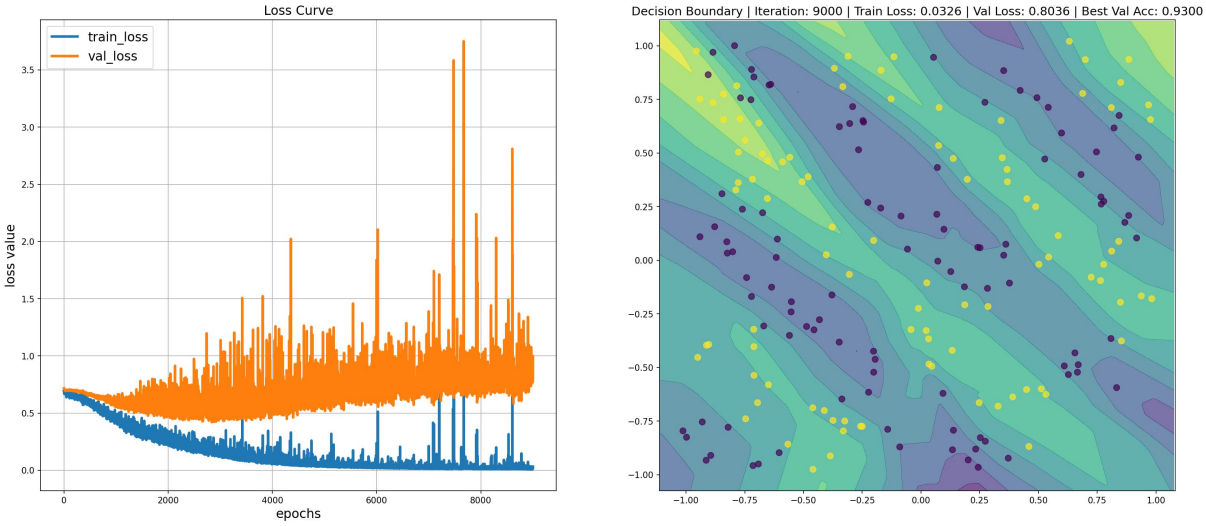

Figure 5: Training results using Vanilla SGD on Sinusoid dataset.

SGD with Momentum

Configs and Training Logs

```
Start training with configs:
  model_config:
    layer_nodes: [2, 32, 16, 16, 1]
    optimizer: SGDWithMomentum(lr=0.01, momentum=0.9)
    hidden_act: relu
  training_config:
    epochs: 9000
    criterion: <class 'loss.BCEFromLogitsLoss'>
  trainset_config:
    data: <ndarray shape=(200, 2)>
    labels: <ndarray shape=(200, 1)>
    batch_size: 64
    shuffle: True
  validation_config:
    count_correct: <function count_binary_correct_from_logits at 0x00000229775789D0>
    mute: True
```

```
  valset_config:
    data: <ndarray shape=(200, 2)>
    labels: <ndarray shape=(200, 1)>
    batch_size: 200
    shuffle: False
MLP(
  (0): Linear(in_features=2, out_features=32)
  (1): ReLU()
  (2): Linear(in_features=32, out_features=16)
  (3): ReLU()
  (4): Linear(in_features=16, out_features=16)
  (5): ReLU()
  (6): Linear(in_features=16, out_features=1)
)
Training done, with highest validation accuracy of 0.94.
```

The best validation accuracy achieved is **94%**.

Visualizations

Figure 6 shows the loss curve and the decision boundary.

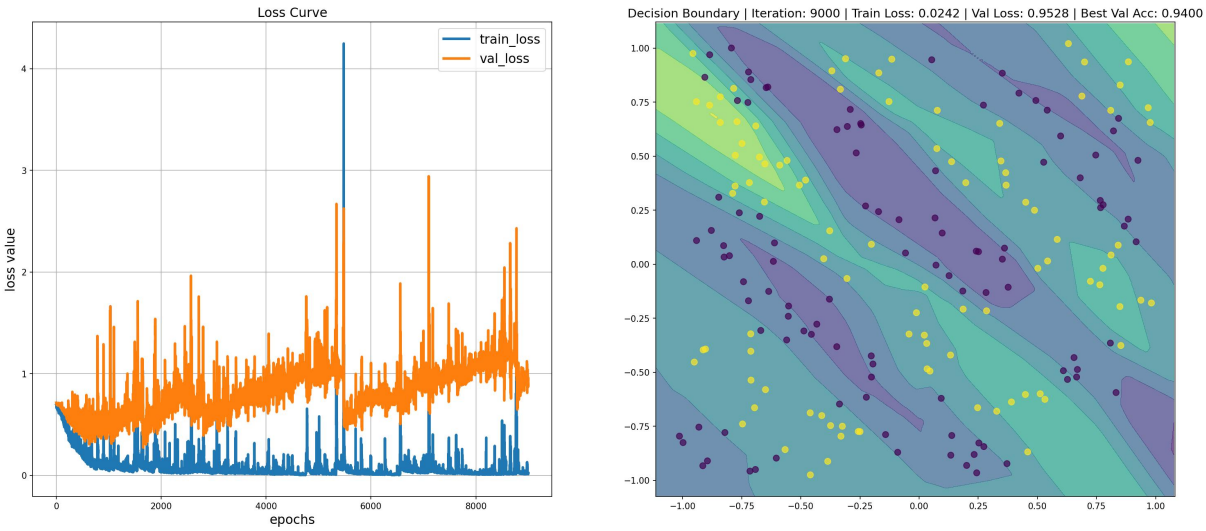

Figure 6: Training results using SGD with momentum on Circle dataset.

Adam

Configs and Training Logs

```
Start training with configs:
  model_config:
    layer_nodes: [2, 32, 16, 16, 1]
    optimizer: Adam(lr=0.02, betas=(0.9, 0.999), eps=1e-09)
    hidden_act: relu
  training_config:
    epochs: 9000
    criterion: <class 'loss.BCEFromLogitsLoss'>
  trainset_config:
    data: <ndarray shape=(200, 2)>
    labels: <ndarray shape=(200, 1)>
```

```
    batch_size: 64
    shuffle: True
  validation_config:
    count_correct: <function count_binary_correct_from_logits at 0x00000229775789D0>
    mute: True
  valset_config:
    data: <ndarray shape=(200, 2)>
    labels: <ndarray shape=(200, 1)>
    batch_size: 200
    shuffle: False
MLP(
  (0): Linear(in_features=2, out_features=32)
  (1): ReLU()
  (2): Linear(in_features=32, out_features=16)
  (3): ReLU()
  (4): Linear(in_features=16, out_features=16)
  (5): ReLU()
  (6): Linear(in_features=16, out_features=1)
)
Training done, with highest validation accuracy of 0.93.
```

The best validation accuracy achieved is **93%**.

Visualizations

Figure 7 shows the loss curve and the decision boundary.

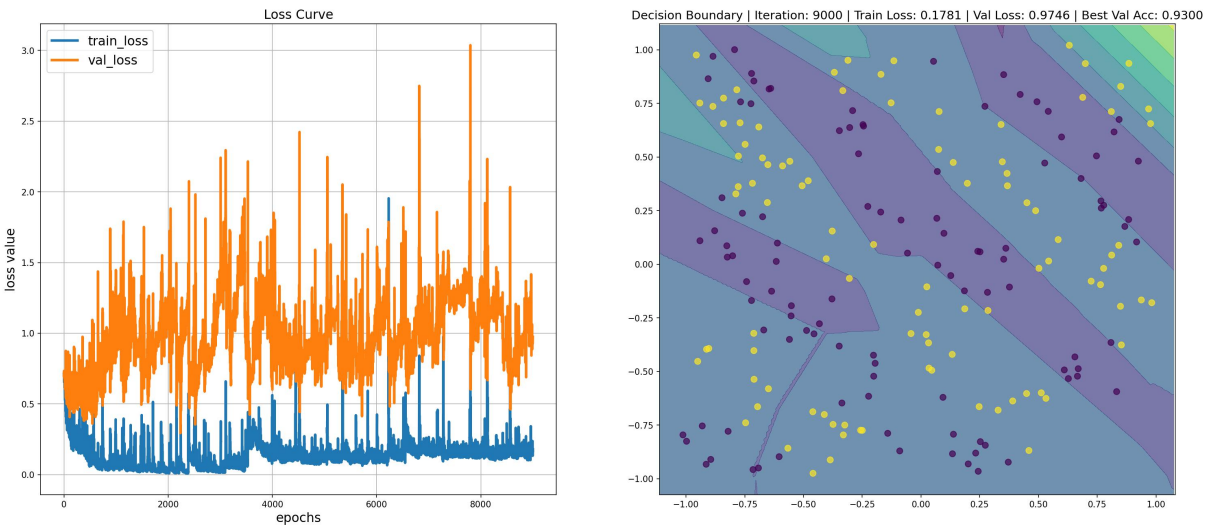

Figure 7: Training results using Adam on Circle dataset.

Analysis

Observation: Though SGDWithMomentum achieved the highest best Validation Accuracy, the Adam optimizer performs best in this task because it has the most smooth and coherent decision boundaries.
Reason: Adam combines momentum-based gradient smoothing with adaptive per-parameter learning rates, which allows Adam to effectively reduce the impact of gradient variance resulting from mini-batch while automatically adjusting the step size according to the local curvature of the loss surface.

Deliverable 6: Swiss Roll

Configs and Training Logs

```
Start training with configs:
  model_config:
    layer_nodes: [2, 32, 16, 16, 1]
    optimizer: Adam(lr=0.005, betas=(0.9, 0.999), eps=1e-09)
    hidden_act: relu
  training_config:
    epochs: 2000
    criterion: <class 'loss.BCEFromLogitsLoss'>
  trainset_config:
    data: <ndarray shape=(200, 2)>
    labels: <ndarray shape=(200, 1)>
    batch_size: 64
    shuffle: True
  validation_config:
    count_correct: <function count_binary_correct_from_logits at 0x000001DAE0DC5940>
    mute: True
  valset_config:
    data: <ndarray shape=(200, 2)>
    labels: <ndarray shape=(200, 1)>
    batch_size: 200
    shuffle: False
MLP(
  (0): Linear(in_features=2, out_features=32)
  (1): ReLU()
  (2): Linear(in_features=32, out_features=16)
  (3): ReLU()
  (4): Linear(in_features=16, out_features=16)
  (5): ReLU()
  (6): Linear(in_features=16, out_features=1)
)
Training done, with highest validation accuracy of 0.995.
```

The best validation accuracy achieved is **99.5%**.

Visualizations

Figure 8 shows the loss curve and the decision boundary.

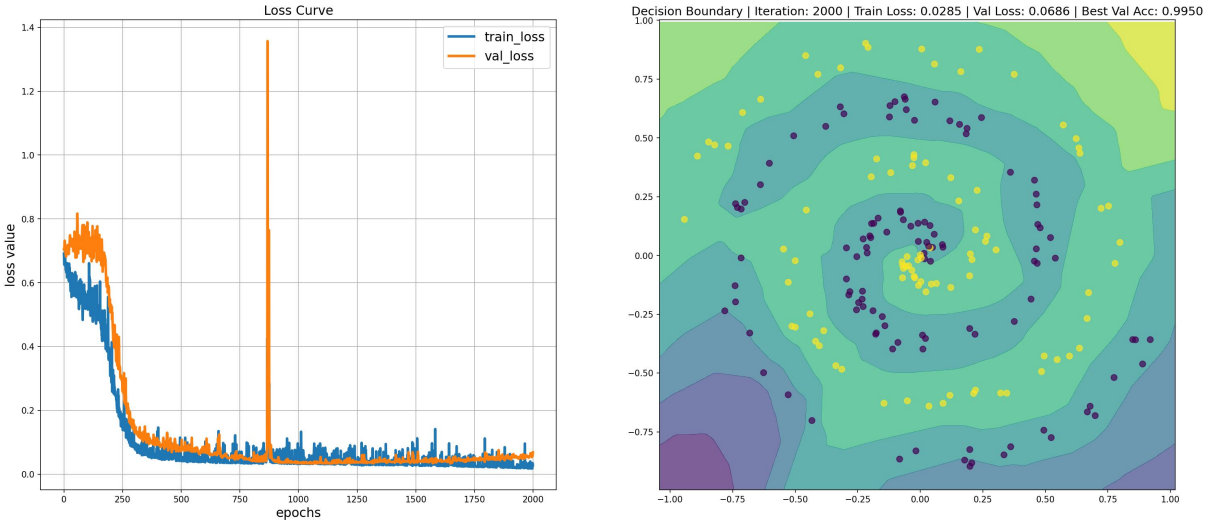

Figure 8: Training results for Swiss Roll dataset.

Deliverable 7: Non-linear Embeddings

XOR Dataset

For the XOR dataset, we introduced the productive feature (xy):

$$\text{Extended Features} = [x, y, xy] \tag{1}$$

Configs and Training Logs

```
Start training with configs:
  model_config:
    layer_nodes: [3, 1]
    optimizer: Adam(lr=0.1, betas=(0.9, 0.999), eps=1e-09)
    hidden_act: relu
  training_config:
    epochs: 2000
    criterion: <class 'loss.BCEFromLogitsLoss'>
  trainset_config:
    data: <ndarray shape=(200, 3)>
    labels: <ndarray shape=(200, 1)>
    batch_size: 64
    shuffle: True
  validation_config:
    count_correct: <function count_binary_correct_from_logits at 0x00000144B1BD8940>
    mute: True
  valset_config:
    data: <ndarray shape=(200, 3)>
    labels: <ndarray shape=(200, 1)>
    batch_size: 200
    shuffle: False
MLP(
  (0): Linear(in_features=3, out_features=1)
)
Training done, with highest validation accuracy of 0.99.
```

The best validation accuracy achieved is **99%**.

Visualizations

Figure 9 shows the loss curve and the decision boundary.

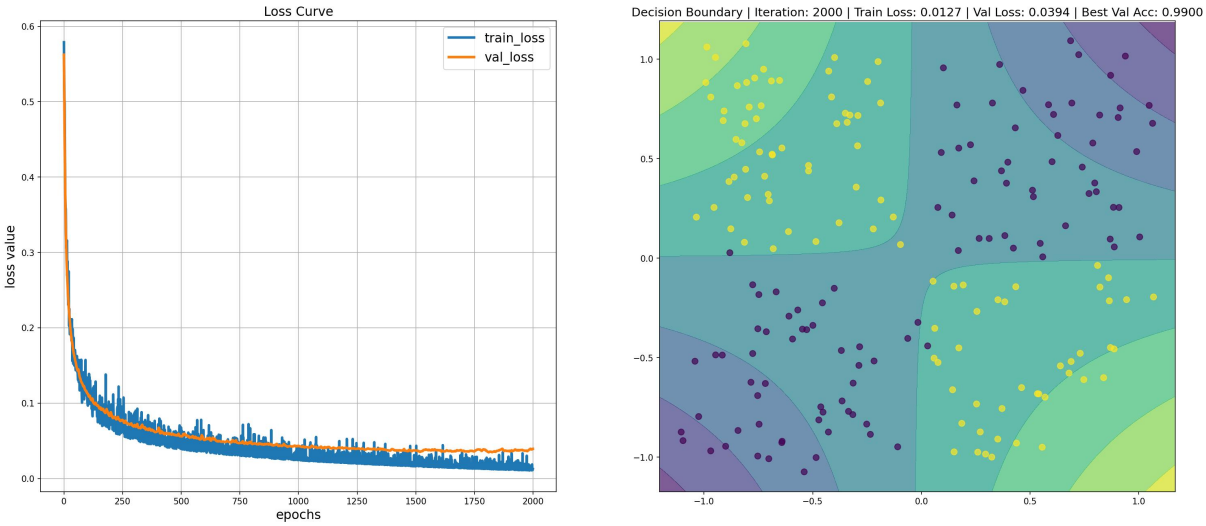

Figure 9: Training results for XOR dataset with feature engineering.

Swiss-roll Dataset

For the Swiss-roll dataset, we introduced the quadratic features (x^2, y^2, xy):

$$\text{Extended Features} = [x, y, x^2, y^2, xy] \tag{2}$$

Configs and Training Logs

```
Start training with configs:
  model_config:
    layer_nodes: [5, 10, 1]
    optimizer: Adam(lr=0.02, betas=(0.9, 0.999), eps=1e-09)
    hidden_act: relu
  training_config:
    epochs: 3000
    criterion: <class 'loss.BCEFromLogitsLoss'>
  trainset_config:
    data: <ndarray shape=(200, 5)>
    labels: <ndarray shape=(200, 1)>
    batch_size: 64
    shuffle: True
  validation_config:
    count_correct: <function count_binary_correct_from_logits at 0x00000144B1BD8940>
    mute: True
  valset_config:
    data: <ndarray shape=(200, 5)>
    labels: <ndarray shape=(200, 1)>
    batch_size: 200
    shuffle: False
MLP(
  (0): Linear(in_features=5, out_features=10)
  (1): ReLU()
  (2): Linear(in_features=10, out_features=1)
```

```
)
Training done, with highest validation accuracy of 0.975.
```

The best validation accuracy achieved is **97.5%**.

Visualizations

Figure 10 shows the loss curve and the decision boundary.

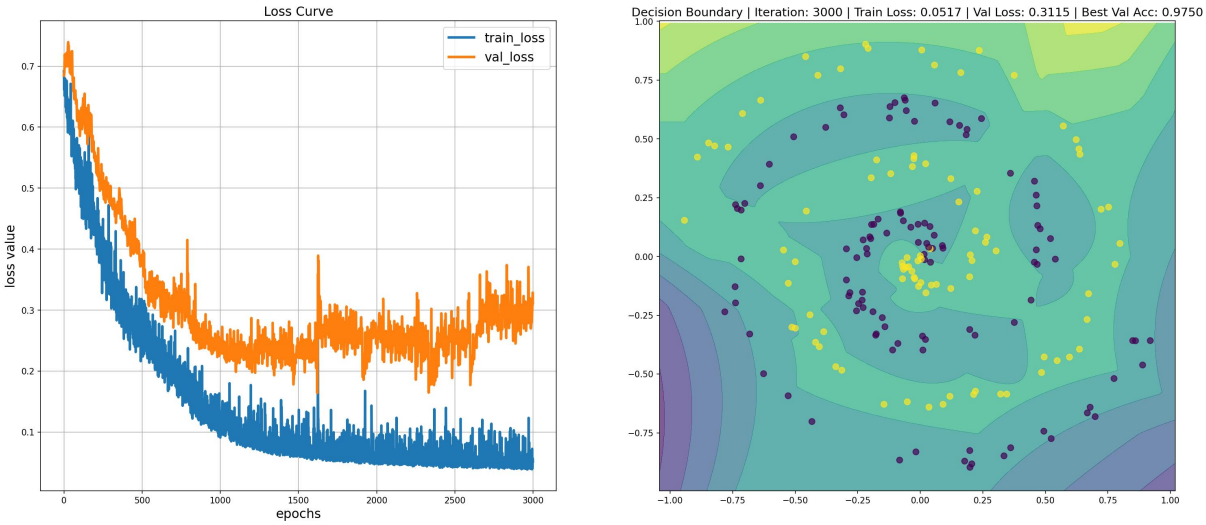

Figure 10: Training results for Swiss-roll dataset with feature engineering.